EE5203 L13 Demonstration Guide

Final demonstration, report, and individual defence - Basri Erdogan

Mission: Demonstrate a working system against requirements you stated in advance, show the evidence for each, trigger a failure on purpose and explain it — then defend your own contribution individually, down to the register.

Prepare before the session

- ☐ **Rehearse the full 15 minutes**, with the hardware, start to finish.
- ☐ **Tag a working commit** and know how to flash it in under two minutes.
- ☐ Bring the report, the requirements table, the resource map, and the timing budget — printed, not only on a laptop.
- ☐ Decide **which fault you will inject**, and rehearse the explanation.
- ☐ Each member: know which parts you wrote and which resources they own.
- ☐ Check the list of "what must be true by L13" agreed at the L12 clinic.

What is assessed

Component	Weight	Team or individual
Demonstration against requirements	35 %	team
Report	25 %	team
Presentation	15 %	team
Individual oral defence	**25 %**	**individual**

A team mark is shared. The defence is not — two members of one team routinely receive different marks, and that is the point of having it.

The demonstration — 15 minutes

Time	Segment	What we look for
0-2	Setup and power-up	it runs before you start explaining
2-7	Requirements walk	one requirement, one test, one observed result
7-12	Live operation + injected fault	the system doing its job, then failing on purpose
12-15	Known issues	specific, honest, with what you know about each

Run the system **first** and talk over it. A demonstration that begins with five minutes of slides before anything is switched on has spent its best minutes.

The injected fault

Choose one and rehearse it:

- disconnect a sensor mid-run and show the system's response;
- hold a breakpoint until `ORE` sets, then show the recovery (or its absence);
- send an out-of-range command and show it refused;
- stall an actuator and show the current or the reset behaviour.

State what you expect **before** you do it. Predicting the failure correctly is worth more than the failure being graceful.

The individual defence — 5 to 8 minutes each

Questions come at three levels:

1. **Your contribution.** "Which parts did you write? Walk me through one."
2. **The mechanism.** "Which register does that configure? What does the hardware do with it?"
3. **The alternative.** "Why this way? What would happen if you did it the other way round?"

Level 3 is where the marks are. Be ready to open the code, open the SFR view, and point.

Acceptance checklist

- ☐ System powered and running within 2 minutes.
- ☐ Every requirement stated, tested, and its result reported.
- ☐ At least one fault injected deliberately, with a prediction stated first.
- ☐ Known-issues list presented, specific and honest.

☐ Report submitted: requirements, decisions, resource map, timing budget, evidence, known issues.
☐ Every member defends their own contribution at register level.
☐ Tagged fallback build available.

Individual oral check — the six questions

Each member should expect three of these:

- Which parts did you write, and which resources do they own?
- Show me where you configure that peripheral. Why those values?
- What is in that register right now, and why?
- What would happen if I removed this line?
- How do you know this requirement is met? Show me the evidence.
- What broke during the term, and how did you find it?

Reflection — submit with the report

Two paragraphs, honest and specific:

1. What would you design differently, knowing what you know now?
2. What cost you the most time, and what would have prevented it?

"Nothing, it went well" scores nothing. Every project has an answer to both.

Submit

- The report (PDF), including requirements, design decisions, resource map, timing budget, evidence, and known issues.
- The presentation slides.
- The final source, with the demonstrated commit tagged.
- The individual reflection, one per member.

If it fails on the day

Do not freeze, and do not start editing code. In order:

1. **State what should have happened.** You know the expected behaviour.
2. **Name the species** — component, resource, or timing.
3. **Use your instruments** — telemetry, counters, the SFR view.
4. **Fall back** to the tagged build and continue the demonstration.

A team that diagnoses a live failure convincingly can score higher than one whose demo worked and who cannot explain why.